

Sunshine Retail Platform

Project Report

Prepared by: Divya Rachala

Abstract

This report documents a full-stack retail project that connects a fifty-product Next.js storefront with Java order processing, Python recommendations, conversational customer support, and reproducible retail analytics. The application covers discovery, cart, checkout, payment outcomes, stock exhaustion, delivery estimates, account views, and recent orders.

Twenty-three automated tests pass across TypeScript, Java, and Python. A synthetic order dataset produces a 75.0 percent confirmation rate, INR 37,136 in confirmed revenue, and INR 4,126.22 average order value. Five experiments examine order outcomes, test coverage, KPI calculations, workflow failures, and application scale.

Keywords: retail platform; Next.js; Java; Python; order workflow; conversational support; testing

Report structure

The report presents the problem, source data, preparation method, evaluation design, five evidence-based experiments, limitations, reproducibility details, and conclusion.

1. Project overview and problem statement

Sunshine is a student-scale retail platform covering product discovery, cart, checkout, payment outcomes, delivery estimates, recent orders, inventory changes, and website-specific conversational support. I built it because many small commerce demonstrations use disconnected data and cannot verify what happens after an error.

The platform connects a Next.js customer interface, a Java Spring Boot order service, a Python FastAPI recommendation service, and a pandas KPI workflow. All products, payments, addresses, and orders are synthetic demonstrations.

2. System design and project boundaries

The customer interface contains fifty products across five retail categories and supports INR pricing, sizes, stock states, delivery rules, account views, and order history. The assistant can answer Sunshine questions, recommend verified products, add a selected option to the cart, and retrieve an exact order.

Java owns typed order validation and the success, payment-failure, and out-of-stock sequence. Python ranks same-category recommendations and produces retail KPIs. Vercel can use contract-compatible adapters when the external services are not configured. The project does not contain production authentication, a real payment gateway, or a shared customer database.

3. Testing methodology

I tested business behaviour at four levels: TypeScript unit tests for pricing, chat, cart and order rules; JUnit tests for Java order outcomes; pytest tests for Python recommendation ranking and validation; and browser captures for storefront, product, checkout, profile, and unavailable-product flows.

The tests target both successful and failed paths. A successful order clears the cart and creates a delivery estimate. A payment failure skips fulfilment and preserves the cart. Stock exhaustion stops before payment. The assistant refuses unrelated questions.

4. Experiment 1: synthetic order outcomes

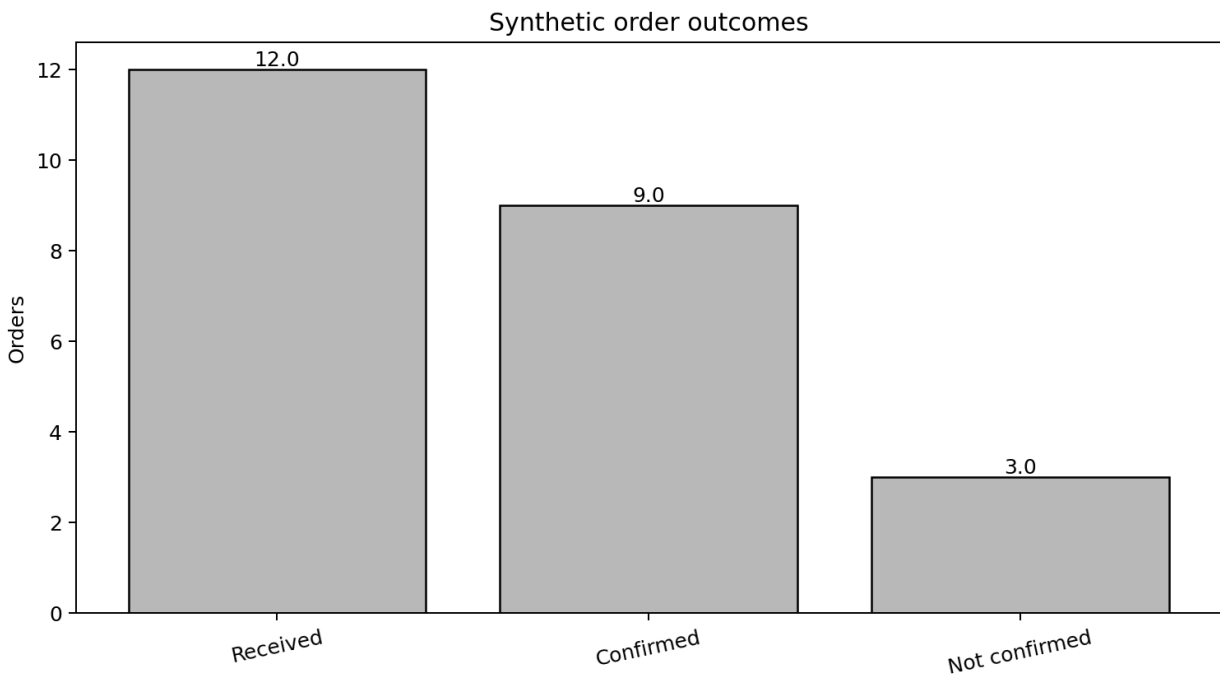


Figure 1. Received, confirmed, and non-confirmed orders in the analytics sample.

What I tested: Whether the pandas workflow calculates order counts and confirmation rate from raw synthetic order rows.

What the graph shows: Nine of 12 orders are confirmed, producing a 75.0 percent confirmation rate.

Conclusion: The analytics output reconciles order status into a clear funnel and keeps failed outcomes visible.

5. Experiment 2: automated test suites

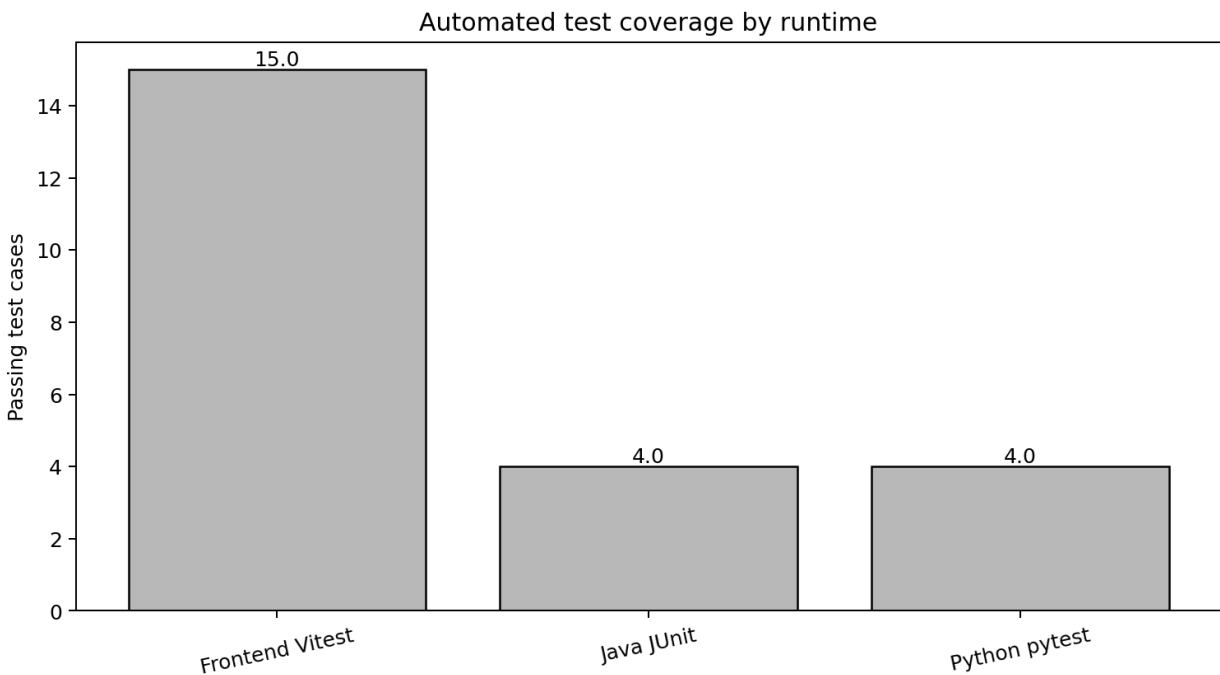


Figure 2. Passing automated tests across the three implementation languages.

What I tested: Pricing, chat scope, stock, payment failure, orchestration, ranking, API responses, and request validation.

What the graph shows: Fifteen frontend tests, four Java tests, and four Python tests pass.

Conclusion: Critical business rules are verified independently in each runtime rather than only through the visible interface.

6. Experiment 3: retail KPI outputs

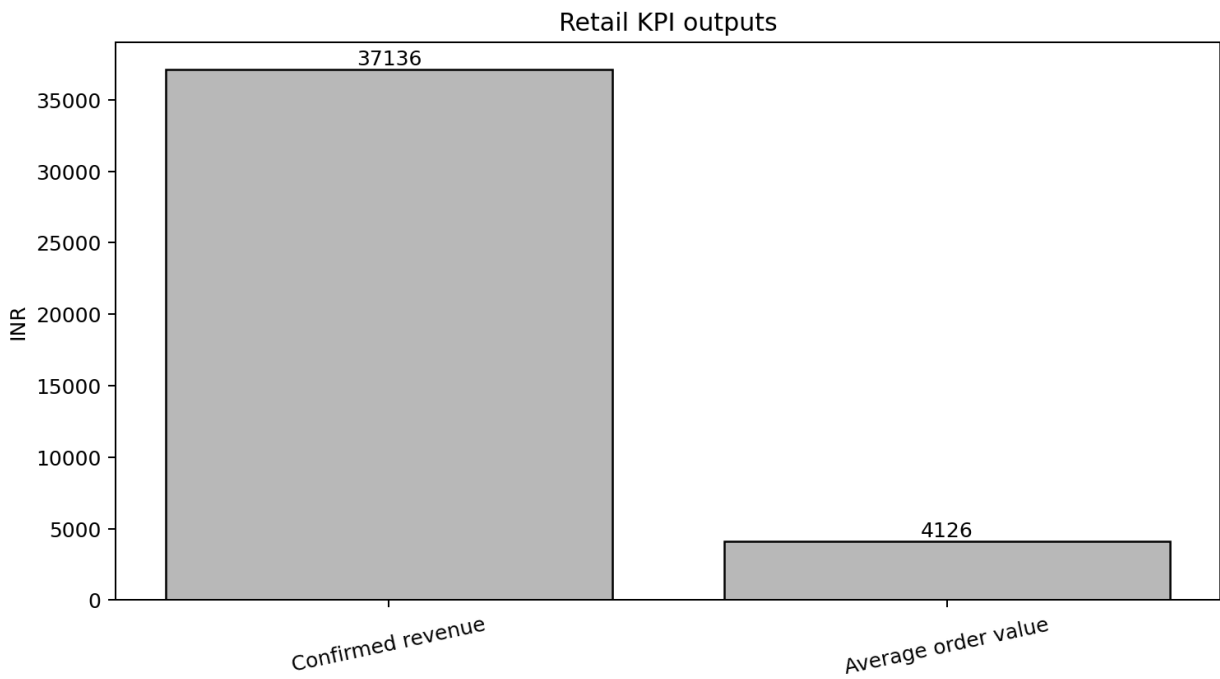


Figure 3. Confirmed revenue and average order value from the reproducible KPI workflow.

What I tested: Whether confirmed-order revenue and average order value are calculated from the synthetic source file.

What the graph shows: Confirmed revenue is INR 37,136 and average order value is INR 4,126.22.

Conclusion: The application includes a measurable data workflow in addition to customer-interface behaviour.

7. Experiment 4: workflow failure paths

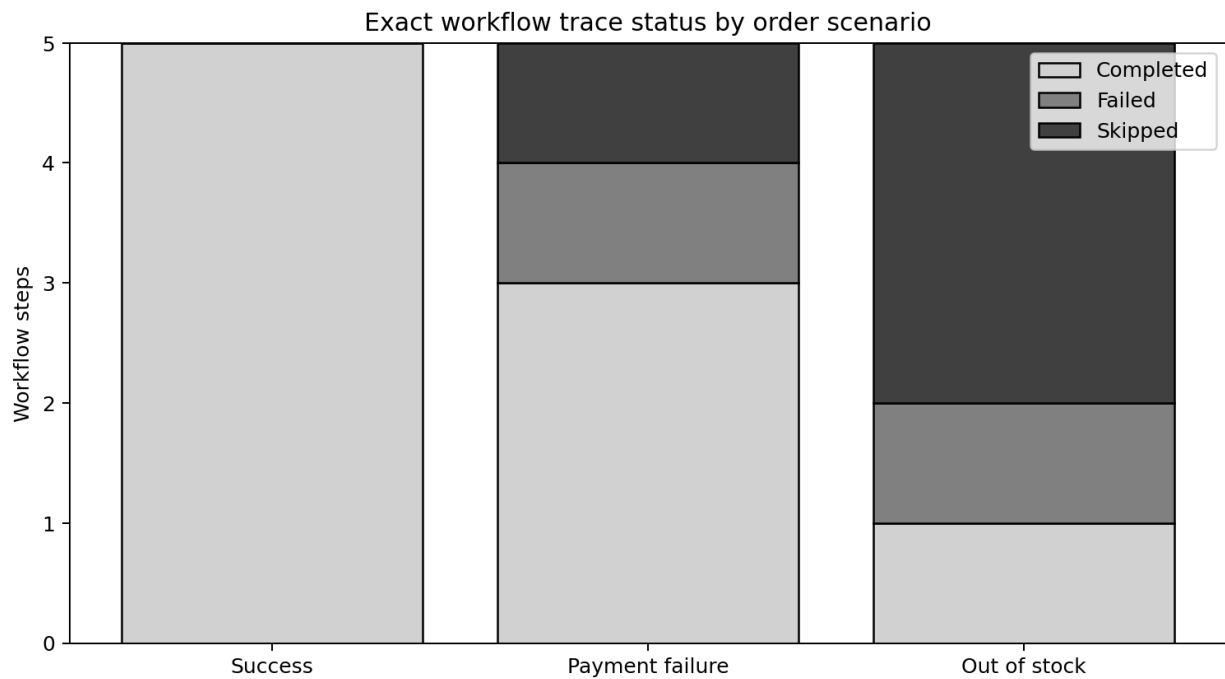


Figure 4. Exact completed, failed, and skipped trace steps for each order scenario.

What I tested: Whether downstream steps stop safely when a prerequisite fails.

What the graph shows: Success records five completed steps. Payment failure records three completed, one failed, and one skipped step. Stock failure records one completed notification, one failed stock check, and three skipped steps.

Conclusion: Failure is an explicit state with a traceable outcome rather than a generic message or silent partial order.

8. Experiment 5: application scale

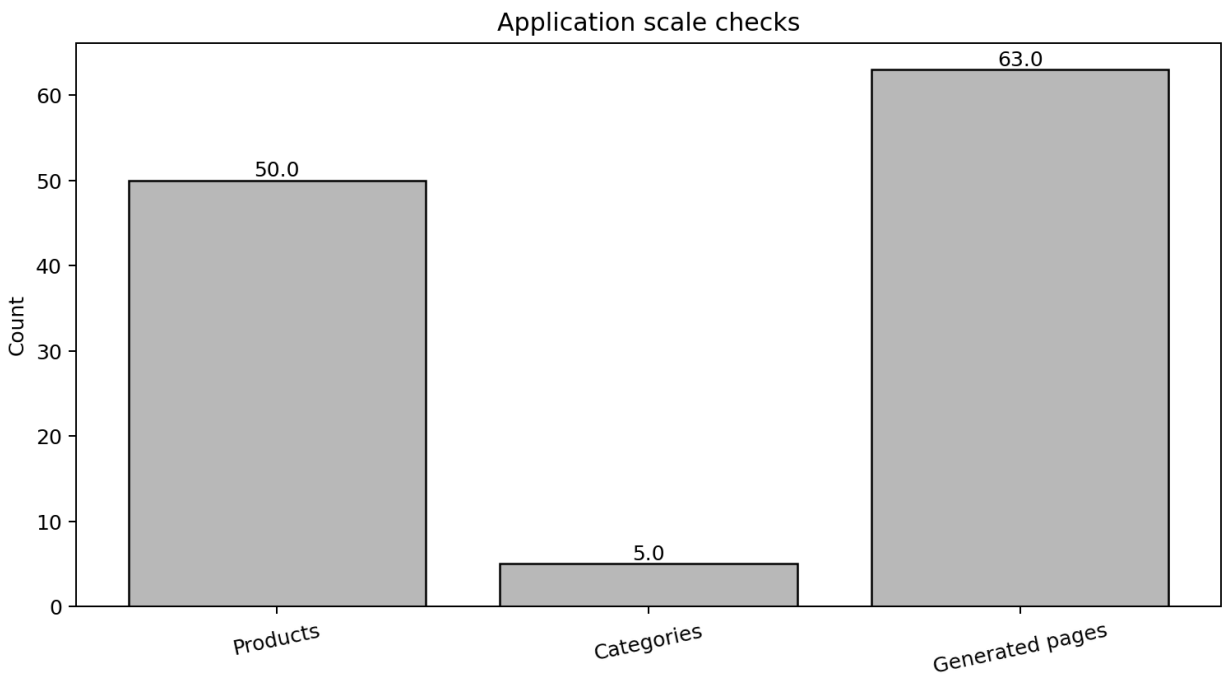


Figure 5. Catalogue and generated-route scale checks.

What I tested: Whether the catalogue and production build contain the intended number of products, categories, and generated pages.

What the graph shows: The build contains 50 products, five categories, and 63 generated pages.

Conclusion: The project is large enough to exercise repeated product and route behaviour while remaining understandable as a student project.

9. Interface evidence and observed behaviour

The following test capture verifies the complete checkout form and outcome controls used for safe demonstration testing. Additional committed captures cover the storefront, product recommendations, profile and recent orders, and a product that becomes unavailable after the final unit is used.

Delivering to Hyderabad 500081

sunshine Search kurtas, shoes, laptops and more

Home Women's fashion Men's fashion Footwear & bags Electronics Home

SECURE DEMO CHECKOUT

Where should we ser

Use sample information only. This portfolio demo does not save your details.

1 Delivery address
Indian PIN codes are supported in this demo.

Full name: Divya Rachala Mobile number: 9876543210

Address: 12, Sunshine Residency, Madhapur

City: Hyderabad PIN code: 500081

Payment method

Figure 6. Checkout test capture showing address, payment method, and demonstration outcome controls.

What I tested: Whether a shopper can proceed from a populated cart through validated Indian address and payment fields.

What the image shows: The checkout keeps customer, delivery, payment, and outcome choices in one understandable sequence.

Conclusion: The interface supports repeatable success and failure testing without processing a real payment.

10. Results, deployment and limitations

The verified build covers search, category discovery, product options, cart totals, free-delivery logic, successful payment, declined payment, unavailable stock, delivery estimates, recent orders, and website-only customer support. The public interface is deployed on Vercel.

Personal cart, order, and inventory state is browser-local. External Java and Python service deployment is optional, and hosted adapters expose which runtime handled the request. The project does not provide real identity, shared stock, fraud detection, courier events, payment settlement, or production security.

11. Reproducibility and future work

The repository includes frontend, Java and Python source, Docker Compose, API contracts, synthetic raw data, KPI outputs, twenty-three automated tests, browser evidence, five evaluation figures, and this report. The complete local stack can be rebuilt without a private production database.

A future version should add authentication, a shared database, idempotent order storage, payment webhooks, inventory transactions, service tracing, and deployment of the real Java and Python services. Those additions should follow only after the current deterministic contracts remain covered by tests.

12. Conclusion

Sunshine demonstrates a complete student retail journey rather than only a storefront screen. The strongest evidence is the connection between UI behaviour, Java failure handling, Python ranking, measurable KPIs, and automated tests. The project remains honest about its boundaries: it is a safe synthetic platform for demonstrating engineering choices, not a production marketplace.